

Leetcode

1. Strings

1. convert string to list: `list(s)`, where `s` is a string.
2. convert list to string: `".join(s)`, where `s` is a list.
3. Ascii format: can use `"b">"a"`: True, `"a">"b"`: False.

2. •

1. list: an ordered sequence (repeats allowed).
2. hashset: a collection of distinct objects where order does not matter.
3. multiset: a collection of objects (repeats allowed) where order does not matter, for example Counter in Python collections.
4. ordered set: a collection of distinct objects where order matters.

3. ordered list is also called **string**, a *word*, or a *sequence*.

4. A list is also called a *sample* or a *selection*, especially in probability and statistics.

5. Lists are sometimes called vectors and the elements components.

6. substring vs subsequence

1. **substring** is a *continuous* part or subpart of a string.
2. **subsequence** is the part of a string or sequence, that might be continuous or not but the *order* of the elements is maintained.
3. Example:
 1. `s="pwwkew"`, `"wke"` is a **substring**, whereas, `"pwke"` is a **subsequence**.

7. subarray

1. subarray is a contiguous sequence in an array
 1. Example: `{1,2,3,4}`
 2. subarray: `{1,2}`, `{1,2,3}`
 3. subsequence: need not be contiguous, but maintains order: `{1,2,4}`
 4. subset: same as subsequence except it has empty set: `{1,3}`, `{}`

8. Patterns:

1. Two pointers
2. Fast and slow pointers
3. Sliding window
4. Merge intervals
5. Cyclic sort

Recursion

1. Write out the code for a recursive function that sums elements in a list.

2. Write a recursive function to count the number of items in a list.

1. Code

```
# -*- coding: utf-8 -*-
"""
Created on Sun Sep 13 11:59:30 2026

@author: reina
"""

"""
A recursive function to count items in a linked list.
"""

class Node():
    def __init__(self, val):
        self.val = val
        self.next = None

class linkedlist():
    def __init__(self, val):
        self.head = Node(val)

    def add(self, val):
        curr_node = self.head
        while curr_node.next != None:
            curr_node = curr_node.next
```

```

curr_node.next=Node(val)

def __len__(self):
    def helper(node):
        if not node:
            return 0
        return 1+helper(node.next)
    return helper(self.head)

def __repr__(self):
    values=[]
    curr_node=self.head
    while curr_node!=None:
        values.append(curr_node.val)
        curr_node=curr_node.next
    return f"{values}"

if __name__ == '__main__':
    ll=linkedlist(2)
    ll.add(3)
    ll.add(4)
    ll.add(11)
    print(ll)
    print(len(ll))

```

3. Write a recursive function to find the maximum number in a list.
4. Binary search is a D&C algorithm, too. Can you come up with a base case and recursive case for binary search?

1. Code:

```

# -*- coding: utf-8 -*-
"""
Created on Sun Sep 13 15:14:48 2026

@author: reina
"""

def binary_search(x,target):
    l,r=0,len(x)-1
    def helper(l,r):
        if l>=r:
            mid=l+(r-l+1)//2
            print(l,mid,r)
            if x[mid]==target:
                return mid
            if target<x[mid]:
                return helper(l,mid-1)
            else:
                return helper(mid+1,r)
        else:
            return -1
    return helper(l,r)

if __name__ == '__main__':
    x=[2,3,5,7,11,13]
    x=[2]
    target=11
    target=3
    res=binary_search(x,target)
    print(res)

```

Sorting

1. Quicksort vs mergesort

1. Quicksort is used more in practice. Quicksort has average time complexity of $O(n \log(n))$ and worst case complexity of $O(n^2)$. Quicksort is an in-place algorithm.
2. mergesort is always $O(n \log(n))$. However, the simple implementation of mergesort is not an in-place algorithm, we need an auxiliary array to merge the two sorted arrays, hence it has $O(n)$ space complexity.
3. mergesort:

1. Code:

```

# -*- coding: utf-8 -*-
"""
Created on Sun Sep 13 13:34:49 2026

@author: reina
"""

def mergesort(x):
    if len(x)<=1:
        return x
    else:
        l,r=0,len(x)-1
        mid=l+(r-l+1)//2

```

```

# mid=len(x)//2
left=mergesort(x[:mid])
right=mergesort(x[mid:])
#merge
res=[]
i,j=0,0
while i<len(left) and j<len(right):
    if left[i]<right[j]:
        res.append(left[i])
        i+=1
    else:
        res.append(right[j])
        j+=1
if i==len(left):
    res.extend(right[j:])
elif j==len(right):
    res.extend(left[i:])
return res

if __name__ == '__main__':
    x=[1,11,3,5,-1,13,2]
    # x=[1,11]
    res=mergesort(x)
    print(res)

```

4. Quicksort

Backtracking

1. Combinations:

1. Code:

```

class Solution(object):
    def combine(self, n, k):
        """
        :type n: int
        :type k: int
        :rtype: List[List[int]]
        """
        res=[]
        track=[]
        def helper(track,depth):
            # Termination condition
            if depth>n+1:
                return
            if len(track)==k:
                res.append(track[:])
                return
            # Make choice: inclusion
            track.append(depth)
            helper(track,depth+1)
            # Undo choice/backtrack
            track.pop()

            # Make choice: exclusion
            helper(track,depth+1)

        helper(track,1)
        return res

if __name__ == '__main__':
    n,k=4,2
    sol=Solution()
    res=sol.combine(n,k)
    print(res)

```

2. Permutations:

1. Code:

```

class Solution(object):
    def permute(self, nums):
        """
        :type nums: List[int]
        :rtype: List[List[int]]
        """
        res=[] # Result
        track=[] # Track
        used=[False]*len(nums) # Choice list
        def helper(track,used): # used here determines the choice list.
            # Termination condition
            if len(track)==len(nums):
                res.append(track[:])
                return
            for i in range(len(nums)):
                if used[i]:
                    continue
                # Make choice

```

```

        used[i]=True
        track.append(nums[i])
        helper(track,used)
        # Undo choice/backtrack
        used[i]=False
        track.pop()

    helper(track,used)
    return res

if __name__ == '__main__':
    nums=[1,2,3]
    sol=Solution()
    res=sol.permute(nums)
    print(res)

```

3. Subsets:

1. Code:

```

class Solution(object):
    def subsets(self, nums):
        """
        :type nums: List[int]
        :rtype: List[List[int]]
        """
        res=[]
        track=[]
        n=len(nums)
        def helper(track,depth):
            # if depth>n:
            #     return
            if depth==n:
                res.append(track[:])
                return
            # Make choice: inclusion
            track.append(nums[depth])
            helper(track,depth+1)
            # Undo choice/backtrack
            track.pop()

            # Make choice: exclusion
            helper(track,depth+1)

        helper(track,0)
        return res

if __name__ == '__main__':
    nums=[1,2,3]
    sol=Solution()
    res=sol.subsets(nums)
    print(res)

```

References:

1. Grokking the Coding Interview, Design Gurus.
2. Grokking Algorithms, Aditya Bhargava.